SORTING - Kỹ thuật xử lý dữ liệu quan trọng

Khóa học C++ Nâng cao

Nội dung bài học

1. Mở đầu: Vai trò của Sắp xếp trong CP
2. 1. Custom Comparator (Hàm Compare)
3. 2. Cấu Trúc Dữ Liệu Pair
4. 3. Struct

Tầm quan trọng của Sắp xếp trong CP

Nền tảng cốt lõi

Sắp xếp là một trong những nền tảng quan trọng nhất trong Competitive Programming (CP).

- Khoảng **60-70%** các bài toán cần tối ưu dữ liệu đều sử dụng sắp xếp như một bước tiền xử lý.
- Là cơ sở để áp dụng các kỹ thuật nâng cao:
  - Hai con trỏ (Two Pointers)
  - Tìm kiếm nhị phân (Binary Search)
  - Tham lam (Greedy)

Sử dụng std::sort cơ bản

Thông thường, ta sử dụng hàm std::sort trong thư viện <algorithm> của C++.

Ví dụ: Sắp xếp tăng dần

```
int a[5] = {5, 2, 3, 9, 7};
std::sort(a, a + 5);
// Mang a sau khi sort: {2, 3, 5, 7, 9}
```

Ví dụ: Sắp xếp giảm dần sử dụng greater<T>()

```
int a[5] = {5, 2, 3, 9, 7};
std::sort(a, a + 5, std::greater<int>());
// Mang a sau khi sort: {9, 7, 5, 3, 2}
```

Vấn đề thực tế

Bài toán đặt ra

Nếu đề bài yêu cầu: *Sắp xếp một mảng sao cho tất cả các số lẻ đứng trước và xếp tăng dần, các số chẵn đứng sau và xếp tăng dần?*

- Lúc này, `greater<int>()` hoàn toàn bất lực!
- Đó là lý do bạn cần làm chủ kỹ thuật **Custom Comparator**.

Mục tiêu bài học

3 mảnh ghép cốt lõi

Qua bài học chuyên sâu này, bạn sẽ làm chủ kỹ thuật sắp xếp dữ liệu cấu trúc cao trong CP qua 3 nội dung:

1. **Hàm Compare (Custom Comparator):** Hiểu rõ bản chất mechanics của hàm sắp xếp để tự định nghĩa mọi quy tắc phức tạp.
2. **Cấu trúc `std::pair`:** Khai thác cơ chế sắp xếp mặc định của Pair để xử lý nhanh các bài toán tọa độ, trọng số.
3. **Cấu trúc `struct`:** Thiết kế các kiểu dữ liệu tùy biến cho các bài toán mô phỏng thực tế và tối ưu hóa bộ nhớ.

1. Custom Comparator (Hàm Compare)

Bản chất của hàm Compare

- Hàm compare (thường đặt tên là `cmp`) là một hàm trả về kiểu dữ liệu `bool`.
- Khi truyền `cmp` vào `std::sort`, bạn đang định nghĩa quy tắc: **"Khi nào thì phần tử A được phép đứng TRƯỚC phần tử B?"**

Cơ chế hoạt động

- Nếu `cmp(A, B)` trả về `true`: A có mức độ ưu tiên cao hơn $B \rightarrow A$ đứng trước B.
- Nếu `cmp(A, B)` trả về `false`: A không có độ ưu tiên cao hơn $B \rightarrow B$ đứng trước hoặc giữ nguyên vị trí.

Ví dụ đầu tiên: Sắp xếp giảm dần

Mục tiêu: Sắp xếp mảng theo thứ tự giảm dần.

```
bool cmp(int a, int b){
    return a > b;
}

sort(a + 1, a + n + 1, cmp);
```

Ý nghĩa của hàm compare

`cmp(A, B) = true`

Nghĩa là:

A **được đứng trước** *B*

Trong ví dụ này:

$$a > b$$

Minh họa cách sort gọi hàm compare

Muốn sắp xếp hai số:

$$5 \qquad 2$$

Minh họa cách sort gọi hàm compare

Muốn sắp xếp hai số:

$$5 \qquad 2$$

Sort hỏi

Có nên để 5 đứng trước 2 không?

```
cmp(5,2)
```

Minh họa cách sort gọi hàm compare

Muốn sắp xếp hai số:

$$5 \qquad 2$$

Sort hỏi

Có nên để 5 đứng trước 2 không?

`cmp(5,2)`

$$5 > 2 \Rightarrow \texttt{true}$$

$$\boxed{5 \text{ đứng trước } 2}$$

Minh họa cách sort gọi hàm compare

Muốn sắp xếp hai số:

$$5 \qquad 2$$

Sort hỏi

Có nên để 5 đứng trước 2 không?

`cmp(5,2)`

$$5 > 2 \Rightarrow \texttt{true}$$

$$\boxed{5 \text{ đứng trước } 2}$$

Lưu ý quan trọng khi viết Compare

Quy tắc cần nhớ

Nếu hai phần tử bằng nhau thì hàm compare phải trả về `false`.

Ví dụ đúng:

```
bool cmp(int a, int b){
    return a > b;
}
```

Khi $a = b$, biểu thức $a > b$ là `false`.

Lưu ý quan trọng khi viết Compare

Quy tắc cần nhớ

Nếu hai phần tử bằng nhau thì hàm compare phải trả về `false`.

Ví dụ sai:

```
bool cmp(int a, int b){
    return a >= b;
}
```

Khi $a = b$, biểu thức $a >= b$ lại là `true`.

Hậu quả

Hàm `sort()` sẽ không biết ai nên đứng trước ai, có thể dẫn đến **Runtime Error**, **TLE** hoặc kết quả không xác định.

Giải quyết bài toán ban đầu: Lẻ trước, chẵn sau

Yêu cầu:

Lẻ trước (tăng dần), chẵn sau (tăng dần).

Ta cần quyết định:

$$\boxed{\texttt{cmp(a,b)} = \texttt{true}}$$

khi nào?

Giải quyết bài toán ban đầu: Lẻ trước, chẵn sau

Yêu cầu:

Lẻ trước (tăng dần), chẵn sau (tăng dần).

Ta cần quyết định:

$$\boxed{\texttt{cmp(a,b)} = \texttt{true}}$$

khi nào?

- Nếu a lẻ, b chẵn $\Rightarrow$ trả về `true`.
- Nếu a chẵn, b lẻ $\Rightarrow$ trả về `false`.
- Nếu cùng chẵn hoặc cùng lẻ $\Rightarrow$ số nhỏ hơn đứng trước.

Hàm Compare

```
bool cmp(int a, int b) {
    if (a % 2 != b % 2)
        return a % 2 > b % 2;

    return a < b;
}
```

Ý tưởng:

- Khác tính chẵn lẻ → số lẻ được ưu tiên.
- Cùng tính chẵn lẻ → sắp xếp tăng dần.

2. Cấu trúc dữ liệu Pair

Cấu trúc dữ liệu std::pair

Trong CP, ta thường quản lý các đối tượng có 2 thuộc tính đi liền nhau:

- Tọa độ X và Y
- Trọng số và Chỉ số đỉnh của đồ thị
- Thời gian bắt đầu và Kết thúc của công việc

`std::pair` giải quyết việc này mà không cần khai báo cấu trúc mới.

Cú pháp cơ bản

```
pair<string, int> student;
student.first = "Duc Cong";
student.second = 2025;

// Khoi tao nhanh
pair<int, int> point = {10, 20};
```

Cơ chế sort mặc định với std::pair

Đây là "vũ khí tối tân" giúp viết code cực nhanh. Mặc định, `std::sort` với `pair` sẽ thực hiện **Sắp xếp theo thứ tự từ điển (Lexicographical order)**:

1. Ưu tiên so sánh thuộc tính `.first` trước.
2. Nếu `.first` hoàn toàn bằng nhau, hệ thống xét tiếp `.second`.

Ví dụ: Sắp xếp mốc thời gian (Giờ, Phút) tăng dần

```
vector<pair<int, int>> times = {{14, 30}, {9, 15}, {14, 15}, {9, 5}};
sort(times.begin(), times.end());

for (auto t : times) {
    cout << t.first << ":" << t.second << " | ";
}
// Result: 9:5 | 9:15 | 14:15 | 14:30 |
```

Mẹo CP: Ép dữ liệu vào `pair<A, B>` để tận dụng `sort()` mặc định, tiết kiệm thời gian viết hàm `cmp`.

3. Struct

Cấu trúc struct

Khi bài toán vượt quá 2 thuộc tính (VD: Thí sinh có Mã số, Họ tên, Điểm Toán, Lý, Hóa), lồng pair (`.first.second.first`) sẽ tạo ra thảm họa cú pháp.
→ **struct** là giải pháp chuyên nghiệp.

Khởi tạo Struct cơ bản

```
struct Point {
    int x, y;
}; // Dung quen dau ; o cuoi struct
```

Kỹ thuật sắp xếp mảng Struct

- `struct` **không có cơ chế so sánh tự động mặc định**.
- Bắt buộc viết hàm `cmp`.
- *Mẹo tối ưu:* Truyền tham số dưới dạng tham chiếu hằng `const &` để tránh copy object vô tội vạ.

Bài toán minh họa Struct

Yêu cầu: Cho danh sách điểm tọa độ Oxy. Sắp xếp tăng dần theo hoành độ X. Nếu X bằng nhau, điểm nào có tung độ Y tăng dần sẽ đứng trước.

```
struct Point {
    int id; // ID theo doi vi tri trong mang ban dau
    int x, y;
};

bool comparePoints(const Point& a, const Point& b) {
    if (a.x != b.x) {
        return a.x < b.x; // Tieu chi 1: tang dan theo x
    }
    return a.y < b.y;      // Tieu chi 2: neu x bang nhau, tang dan theo y
}
```

Ứng dụng Struct trong main

```
int main() {
    vector<Point> plist = {
        {1, 5, 10}, // P1: (5, 10)
        {2, 2, 8},  // P2: (2, 8)
        {3, 5, 3},  // P3: (5, 3)
        {4, 2, 1},  // P4: (2, 1)
        {5, 9, 4}   // P5: (9, 4)
    };

    // Sap xep struct voi ham compare da viet
    sort(plist.begin(), plist.end(), comparePoints);

    cout << "ID\tX\tY" << endl;
    for (const auto& p : plist) {
        cout << p.id << "\t" << p.x << "\t" << p.y << endl;
    }
    return 0;
}
```

Giải thích luồng xử lý dữ liệu (Trace)

- **Xét nhóm có $X = 2$:** P2 $(2, 8)$ và P4 $(2, 1)$. Vì X bằng nhau, xét tiếp tung độ Y. Lúc này $1 < 8$, do đó P4 đứng trước P2.
- **Xét nhóm có $X = 5$:** P1 $(5, 10)$ và P3 $(5, 3)$. Tương tự, hoành độ bằng nhau nên xét tung độ Y. Vì $3 < 10$, P3 đứng trước P1.
- **Xét nhóm có $X = 9$:** P5 $(9, 4)$ có hoành độ lớn nhất toàn cục nên bị đẩy xuống cuối danh sách.

Nhận xét tổng quan

Cột X đã được sắp xếp tăng dần ($2 \to 2 \to 5 \to 5 \to 9$). Tại các điểm gãy nơi X trùng nhau, cột Y đóng vai trò quyết định thứ tự. Đây là tư duy cốt lõi trong CP.

Kết quả sau khi sắp xếp

ID	X	Y
4	2	1
2	2	8
3	5	3
1	5	10
5	9	4